Documento Técnico — Frontend Intranet

0. Overview de la intranet

Funciones

Gestión de usuarios
- Utilizar servicio externo (por ahora AD como provider)
- Crear usuarios
- Ver usuarios
- Auth con usuarios

Tareas o incidentes
- TODO list
- Crear nuevas tareas
- Asignar tareas a usuarios

Maquinas y servidores
- Ver estado de maquinas
- Ver servicios de dicha maquina
- Ver información adicional (SO, ip...)

Integración con IA
- Analizar toda la infromación anterior mediante un chatbot

Páginas

Login
- Iniciar sesión con usuarios del dominio

Dashboard principal
- Ver auth user
- Overview de los servidores
- Proximas tareas para el usuario
- Hablar con el chatbot

Monitor de servidores
- Lista de maquinas con su estado
- Añadir/eliminar maquias

Gestión de usuarios
- Lista de usuarios
- Añadir/eliminar usuarios

API (overview)

Auth
- POST /auth/login
- GET /auth/me
- POST /auth/logout

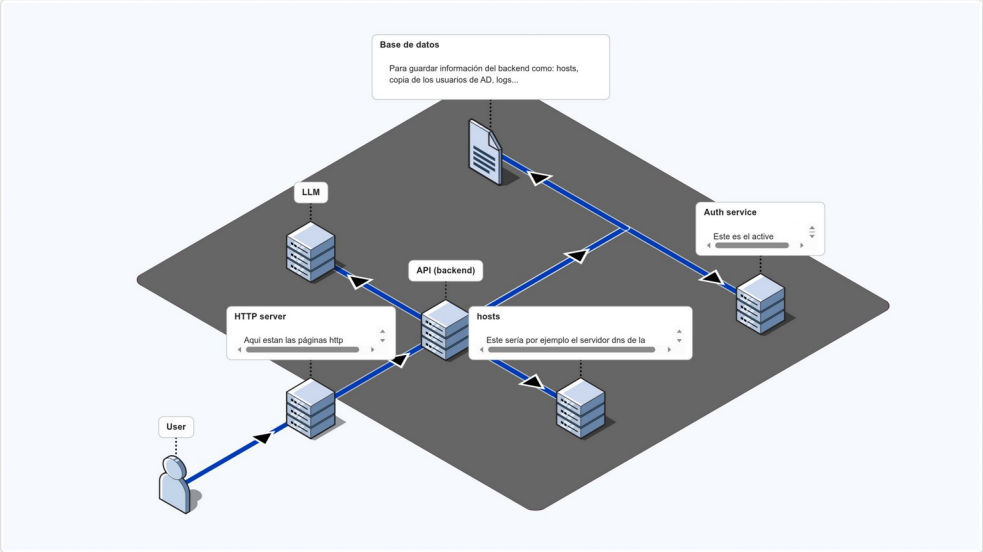
Gestión de usuarios
- GET /api/users
- GET /api/users/{:id}
- POST /api/users

Monitor de servidores
- GET /api/hosts
- GET | UPDATE | DELETE /api/hosts/{:id}
- POST /api/hosts

Tareas
- GET /api/tasks [query by user]
- POST /api/tasks
- UPDATE | DELETE /api/tasks/{:id}

IA
- POST /api/llm/new-message

Arquitecutura



1. Objetivo del frontend

El frontend es la parte visual de la aplicación. Se encarga de mostrar la interfaz, enviar datos al backend y mostrar la respuesta.

2. Tecnologías

HTML: estructura de la página.

Tailwind CSS: estilos.

TypeScript: lógica (similar a JavaScript pero más seguro).

Vite: herramienta que ejecuta y construye el proyecto.

3. Funcionamiento general

```
HTML → TypeScript → API → Respuesta → Renderizado
```

4. Estructura del proyecto

```
frontend/
│
├── index.html
├── dashboard.html
├── users.html
│
├── /src/
│   ├── controllers/
│   ├── services/
│   ├── components/
│   ├── styles/
```

5. Ejecución del proyecto

```
npm install

npm run dev
```

6. Ejemplo Login

```
<form id="loginForm">
  <input name="email" />
  <input name="password" type="password" />
  <button>Login</button>
</form>
```

```
const form = document.querySelector("#loginForm");

form.addEventListener("submit", async (e) => {
  e.preventDefault();

  const data = new FormData(form);

  const res = await fetch("/api/auth/login", {
    method: "POST",
    body: JSON.stringify({
      email: data.get("email"),
      password: data.get("password"),
    }),
    headers: {
      "Content-Type": "application/json",
    },
  });

  const json = await res.json();

  localStorage.setItem("token", json.token);

  window.location.href = "/dashboard.html";
});
```

7. Comunicación con backend

Se usa fetch("/api/...") para comunicarse con el backend.

8. Guardar sesión

```
localStorage.setItem("token", token);
```

9. Protección de rutas

```
if (!localStorage.getItem("token")) {
  window.location.href = "/index.html";
}
```

10. Renderizado de datos

```
container.innerHTML = users.map(u => `
  <tr>
    <td>${u.name}</td>
    <td>${u.role}</td>
  </tr>
`).join("");
```

11. Build y entrega

```
npm run build
```

Subir únicamente la carpeta /dist al servidor.

12. Errores comunes

- No usar type="module"

- Rutas incorrectas

- No usar await

13. Resumen

HTML = estructura, Tailwind = estilo, TypeScript = lógica, Vite = herramienta, Backend = datos.